

C++ Casts — Ամբողջական ուղեցույց

Junior / Mid C++ հարցազրույցի խորացված ուղեցույց

Այս ուղեցույցը մանրամասն բացատրում է C++-ի բոլոր cast-երը՝ `static_cast`, `dynamic_cast`, `const_cast`, `reinterpret_cast` և C-style cast: ինչ է անում յուրաքանչյուրը, երբ է compile error տալիս, երբ ոչ, երբ է runtime-ում ձախողվում, և C-style cast-ը իրականում որ cast-ի է վերածվում:

0. Ինչո՞ւ 4 տեսակ cast

C-ում կար միայն մեկ ձև՝ `(type)value` : Խնդիրն այն էր, որ այդ մեկ ձևը **ամեն ինչ անում էր լուռ**՝ և անվտանգ, և վտանգավոր փոխակերպումներ, առանց տարբերության:

C++-ը բաժանեց 4 առանձին cast-ի, որ.

1. Կողը կարդալիս **երևա մտադրությունը** (ինչ էս ուզում անել):
2. Compiler-ը կարողանա **բռնել սխալ** cast-երը compile time-ում:
3. `grep` -ով հեշտ լինի գտնել վտանգավոր cast-երը (`reinterpret_cast` , `const_cast`):

<code>static_cast</code>	-> «նորմալ», ստուգված փոխակերպումներ (compile-time)
<code>dynamic_cast</code>	-> polymorphic տիպերի runtime-ստուգված downcast
<code>const_cast</code>	-> const/volatile ավելացնել կամ հեռացնել
<code>reinterpret_cast</code>	-> bit-level վերաիմաստավորում (ամենավտանգավորը)

1. static_cast

Ինչ է անում

`static_cast<T>(x)` անում է **compile-time** փոխակերպում իրար հետ «կապ ունեցող» տիպերի միջև: Compiler-ը ստուգում է, որ փոխակերպումը իմաստ ունի; եթե ոչ՝ `compile error`: Runtime-ում ոչ մի ստուգում չկա:

Case 1 — թվային տիպեր (OK)

```
double d = 3.9;
int i = static_cast<int>(d);    // i == 3 (կտրում է կոտորակը)

int a = 7, b = 2;
double r = static_cast<double>(a) / b;    // 3.5, ոչ 3
```

Case 2 — void -> T (OK)

```
void* p = malloc(sizeof(int));
int* ip = static_cast<int*>(p);    // OK
```

Case 3 — Upcast: Derived -> Base (OK, միշտ անվտանգ)

```
class Base {};
class Derived : public Base {};

Derived d;
Base* b = static_cast<Base*>(&d);    // OK (upcast միշտ ապահով է)
```

Case 4 — Downcast: Base -> Derived (կոմպիլվում է, ԲԱՅՑ ստուգում չկա)

```
Base* b = new Derived();
Derived* d = static_cast<Derived*>(b);    // OK, եթե իրականում Derived է

Base* b2 = new Base();
Derived* d2 = static_cast<Derived*>(b2); // ԿՈՄՊԻԼԵՏԻՄ Է, բայց UNDEFINED BEHAVIOR
```

Սա կարևոր տարբերությունն է `dynamic_cast`-ից. `static_cast`-ը downcast-ի ժամանակ **չի ստուգում** իրական տիպը, ուստի սխալ դեպքում undefined behavior է (crash կամ աղբ):

Երբ `static_cast`-ը COMPILE ERROR է տալիս

```
int i = 10;
char* p = static_cast<char*>(&i);    // ERROR: int* -> char* կապ չունեն
```

```
class A {};
class B {};    // իրար հետ կապ չունեն (ոչ inheritance)
A a;
B* b = static_cast<B*>(&a);    // ERROR: unrelated types
```

```
const int x = 5;
int* p = static_cast<int*>(&x);    // ERROR: static_cast-ը const չի հեռացնում
```

Ամփոփ

```
static_cast:
  OK    -> թվային փոխակերպումներ, void*<->T*, upcast, հարաբերվող տիպեր
  ERROR -> unrelated pointer types, const-ի հեռացում
  UB    -> սխալ downcast (կոմպիլվում է, բայց runtime-ում վտանգավոր)
```

2. dynamic_cast

Ինչ է անում

`dynamic_cast<T>(x)` անում է **runtime-ստուգված** cast, հիմնականում **downcast**-ի համար (Base -> Derived): Օգտագործում է RTTI-ն և ստուգում՝ object-ի իրական տիպն իսկապես համապատասխանում է:

Կարևոր պայման. class-ը պետք է լինի **polymorphic** (զոնե մեկ virtual function):

Case 1 — հաջող downcast

```
class Animal { public: virtual ~Animal() {} };
class Dog : public Animal { public: void bark() {} };

Animal* p = new Dog();
Dog* d = dynamic_cast<Dog*>(p);    // հաջող -> valid pointer
if (d) d->bark();                  // աշխատում է
```

Case 2 — ձախողված downcast (pointer -> nullptr)

```
class Cat : public Animal {};
```



```
Animal* p = new Cat();
Dog* d = dynamic_cast<Dog*>(p);    // p-ի տակ Cat է -> nullptr
if (!d) cout << "not a Dog";      // ապահով ստուգում
```

Case 3 — reference-ի դեպքում -> exception (ՈՉ nullptr)

```
Animal* p = new Cat();
try {
    Dog& d = dynamic_cast<Dog&>(*p);    // ձախողում -> throw
} catch (const std::bad_cast& e) {
    cout << "bad_cast: " << e.what();
}
```

Reference-ը չի կարող լինել `nullptr`, ուստի ձախողման դեպքում `dynamic_cast` -ը `std::bad_cast` է throw անում:

Երբ `dynamic_cast`-ը COMPILE ERROR է տալիս

```
class A {};           // ՈՉ polymorphic (virtual չկա)
class B : public A {};

A* a = new B();

B* b = dynamic_cast<B*>(a);  // ERROR: 'A' is not polymorphic
```

Լուծումը՝ `A`-ին ավելացնել `virtual ~A() {}`, և `dynamic_cast`-ը կաշխատի:

`static_cast` vs `dynamic_cast` (downcast)

```
Base* b = new Base();

Derived* d1 = static_cast<Derived*>(b);  // կոմպիլվում է, UB (ստուգում չկա)
Derived* d2 = dynamic_cast<Derived*>(b);  // կոմպիլվում է, d2 == nullptr (ապահով)
```

Ամփոփ

```
dynamic_cast:
  պայման -> polymorphic class (virtual function)
  pointer ծախողում -> nullptr
  reference ծախողում -> throw std::bad_cast
  ERROR -> ոչ polymorphic տիպ
  դաժնալ է static_cast-ից (runtime RTTI ստուգում)
```

3. const_cast

Ինչ է անում

`const_cast<T>(x)` -ը միակ `cast`-ն է, որ կարող է **ավելացնել կամ հեռացնել** `const` (կամ `volatile`) որակիչը: Չի փոխում `bit`-երը, միայն `type system`-ի `const`-ությունը:

Case 1 — const հեռացնել (կանչելու համար)

```
void legacy(char* s);    // հին API, const չի ընդունում

void modern(const char* s) {
    legacy(const_cast<char*>(s));    // հեռացնում ենք const-ը կանչի համար
}
```

Case 2 — ԵՐԲ Է UNDEFINED BEHAVIOR

Ամենակարևոր կանոնը. `const`-ը հեռացնելը **անվտանգ է միայն, եթե original object-ը իրականում `const` չէր**:

```
int x = 10;
const int* p = &x;    // x-ը իրականում const ՉԷ
int* q = const_cast<int*>(p);
*q = 20;    // OK, x-ը հիմա 20 է
```

```
const int y = 10;    // y-ը ԻՍԿԱՊԵՍ const է
int* q = const_cast<int*>(&y);
*q = 20;    // UNDEFINED BEHAVIOR (crash կամ անարդյունք)
```

Երբ const_cast-ը COMPILE ERROR է տալիս

```
int x = 10;
double* d = const_cast<double*>(&x);    // ERROR: const_cast-ը type չի փոխում, միայն c
```

`const_cast` -ը **չի կարող** փոխել իրական տիպը (`int` -> `double`): Այն միայն `const`/`volatile` է ավելացնում/հեռացնում:

Ամփոփ

const_cast:

- OK -> const/volatile ավելացնել/հեռացնել
 - ERROR -> իրական տիպ փոխել (int* -> double*)
 - UB -> իսկապես const object-ի փոփոխում
-

4. reinterpret_cast

Ինչ է անում

`reinterpret_cast<T>(x)` -ը ամենավտանգավոր cast-ն է, bit-երը վերաիմաստավորում է որպես բոլորովին այլ տիպ: Ոչ մի փոխակերպում, ոչ մի ստուգում՝ պարզապես «նույն bit-երը, նոր տիպ»:

Case 1 — pointer -> integer

```
int x = 10;
int* p = &x;
uintptr_t addr = reinterpret_cast<uintptr_t>(p); // հասցեն որպես թիվ
int* p2 = reinterpret_cast<int*>(addr);          // հեն
```

Case 2 — unrelated pointer types

```
struct A { int x; };
struct B { int y; };

A a{5};
B* b = reinterpret_cast<B*>(&a); // կոմպիլվում է, b->y-ը a.x-ի bit-երն են
```

Case 3 — byte-level ընթերցում

```
float f = 3.14f;
unsigned char* bytes = reinterpret_cast<unsigned char*>(&f); // float-ի bytes
```

Երբ reinterpret_cast-ը COMPILE ERROR է տալիս

```
double d = 3.14;
int i = reinterpret_cast<int>(d); // ERROR: չի աշխատում ոչ-pointer արժեքների վրա այ
```

```
const int x = 5;
int* p = reinterpret_cast<int*>(&x); // ERROR: const-ը չի կարող հեռացնել
```

`reinterpret_cast` -ը **չի կարող հեռացնել `const`-ը** (դա `const_cast`-ի գործն է), և չի աշխատում թվային `value`-ից `value` փոխակերպման համար (օրինակ `double -> int`):

Ամփոփ

`reinterpret_cast`:

OK -> `pointer<->integer`, unrelated pointer types, byte reinterpretation

ERROR -> `const`-ի հեռացում, թվային `value->value` (`double->int`)

UB -> strict aliasing-ի խախտում, սխալ տիպով ընթերցում

5. C-style cast (type)value

Ինչ է անում

(T)x հին, C-ից եկած ձևն է: Իրականում compiler-ը այն **թարգմանում է** C++ cast-երի հաջորդականության. փորձում է հերթով և ընտրում առաջին աշխատողը.

(T)x -> compiler-ը փորձում է այս հերթականությամբ:

1. const_cast
2. static_cast
3. static_cast + const_cast
4. reinterpret_cast
5. reinterpret_cast + const_cast

Case 1 — static_cast-ի պես

```
double d = 3.9;
int i = (int)d;    // = static_cast<int>(d) -> 3
```

Case 2 — const_cast-ի պես

```
const int x = 5;
int* p = (int*)&x;    // = const_cast (const հեռացնում է)
```

Case 3 — reinterpret_cast-ի պես (վտանգավոր, լուռ)

```
int x = 10;
double* d = (double*)&x;    // = reinterpret_cast -> վտանգավոր, բայց լուռ կոմպիլվում
```

Ինչո՞ւ C-style cast-ը վատ է

```
int* p = (int*)somePointer;
```

Այս մեկ տողից չես կարող ասել՝ static, const, թե reinterpret cast է կատարվում: Այն կարող է **լուռ** անել ամենավտանգավոր **reinterpret_cast**-ը, երբ դու ուզում էիր անվտանգ

`static_cast` : Դրա համար C++-ում խորհուրդ է տրվում միշտ օգտագործել **named cast**-երը:

Հատուկ դեպք — C-style cast-ը ԿԱՐՈՂ է անել այն, ինչ `static_cast`-ը չի կարող

```
class Base {};  
class Derived : private Base {}; // private inheritance  
  
Derived d;  
Base* b1 = static_cast<Base*>(&d); // ERROR (private base անհասանելի)  
Base* b2 = (Base*)&d; // ԿՈՄՊԻԼՎՈՒՄ Է (C-cast-ը շրջանցում է access-ը)
```

Սա էլ պատճառ է խուսափելու C-style cast-ից. այն կարող է շրջանցել access control-ը՝ լուռ:

6. Համեմատական աղյուսակ

Cast	Compile-time check	Runtime check	const փոխել	Հիմնական օգտագործում
static_cast	Այո	Ոչ	Ոչ	թվային, upcast, void*
dynamic_cast	Այո (polymorphic)	Այո (RTTI)	Ոչ	ապահով downcast
const_cast	Այո	Ոչ	Այո	const ավելացնել/ հեռացնել
reinterpret_cast	Նվազ	Ոչ	Ոչ	bit-level, pointer<->int
C-style (T)	Կախված	Ոչ	Այո	(խուսափել)

7. «Ո՞ր դեպքում ինչ» — արագ որոշման ուղեցույց

Թվային փոխակերպում (double->int)	-> static_cast
Upcast (Derived* -> Base*)	-> static_cast (կամ implicit)
Downcast, ապահով (Base* -> Derived*)	-> dynamic_cast
void* -> T*	-> static_cast
const հեռացնել (legacy API)	-> const_cast
pointer -> integer / bit-level	-> reinterpret_cast
չգիտես որը -> Մի օգտագործիր C-style cast	

8. Compile Error vs OK vs UB — ամփոփ թեստ

```
double d = 3.9;
int a = static_cast<int>(d);           // OK -> 3

int x = 10;
char* c = static_cast<char*>(&x);      // COMPILE ERROR (unrelated)

const int y = 5;
int* p = static_cast<int*>(&y);         // COMPILE ERROR (const)
int* q = const_cast<int*>(&y);          // OK կոմպիլ, բայց *q=... -> UB

class A {};                             // ոչ polymorphic
A* pa = new A();
A* pb = dynamic_cast<A*>(pa);           // OK (նույն տիպ, բայց dynamic-ի իմաստ չկա)
class P { virtual ~P(){} }; class Q:public P {};
P* pp = new P();
Q* qq = dynamic_cast<Q*>(pp);           // OK կոմպիլ, qq == nullptr (ապահով)
Q* qs = static_cast<Q*>(pp);            // OK կոմպիլ, UB (ստուգում չկա)

int n = 10;
double* dp = reinterpret_cast<double*>(&n); // OK կոմպիլ, *dp ընթերցելը -> UB
int val = reinterpret_cast<int>(3.14);    // COMPILE ERROR
```

Cheat Sheet

static_cast — ստուգված compile-time փոխակերպումներ (թվային, upcast, void*): Downcast-ը կոմպիլվում է բայց չի ստուգվում -> UB սխալ դեպքում:

dynamic_cast — runtime-ստուգված downcast polymorphic տիպերի համար: pointer ձախողում -> nullptr, reference ձախողում -> `std::bad_cast`: Ոչ polymorphic տիպ -> compile error:

const_cast — միակ cast-ը, որ const/volatile ավելացնում/հեռացնում է: Իսկապես const object-ի փոփոխումը -> UB: Type չի փոխում:

reinterpret_cast — bit-level վերաիմաստավորում (pointer<->int, unrelated pointers): Ամենավտանգավորը, const չի հեռացնում, value->value չի անում:

C-style (T)x — վերածվում է const/static/reinterpret cast-ի հաջորդականության; լուռ կարող է անել վտանգավոր փոխակերպում: խուսափիր դրանից, օգտագործիր named cast-երը:

OneMarketData takeaways

static_cast	-> compile-time, no runtime check, downcast=UB if wrong
dynamic_cast	-> runtime check, needs virtual, fail=nullptr/bad_cast
const_cast	-> only const/volatile, UB on truly-const object
reinterpret_cast	-> raw bits, most dangerous, no const removal
C-style cast	-> picks one of the above silently -> avoid